ME5406 - DEEP LEARNING FOR ROBOTICS

NATIONAL UNIVERSITY OF SINGAPORE

COLLEGE OF DESIGN AND ENGINEERING

Project 2: Reinforcement Learning for a Simplified Robotic Grasping Task

Author:
PAN YAN (ID: A0318887H)

1 Introduction

Robotic grasping is a core capability needed across a wide range of real-world applications, from industrial assembly and warehouse automation to assistive and surgical robotics. Despite decades of research, reliable autonomous grasping in unstructured environments remains an open problem. Traditional approaches such as analytical grasp planning and sampling-based motion planners like RRT offer strong theoretical guarantees — probabilistic completeness, sub-millimetre accuracy, and immediate applicability given a kinematic model. In practice, however, they depend on precise geometric knowledge of both the robot and the object. Even a small localisation error of 1–2 cm can cause a grasp to miss entirely. Furthermore, a new collision-free trajectory must be computed from scratch for every new object position, and repeated failures provide no information that could improve future attempts.

Deep learning methods, including convolutional neural network (CNN) grasp detectors [1, 2] and end-to-end visuomotor policies [3], have shown strong generalisation from raw sensor data. However, supervised approaches require large labelled datasets [4], and imitation learning is constrained by the quality of the expert demonstrations used.

Reinforcement learning (RL) offers a different approach [5]: an agent learns a grasp policy entirely through trial-and-error interaction with the environment, without needing an explicit dynamics model or labelled data. In this project, a UR5 robotic arm with a Robotiq 85 two-finger gripper is trained in a PyBullet physics simulation to locate and grasp a randomly placed cube on a table surface.

Two state-of-the-art off-policy RL algorithms are implemented and compared: Soft Actor-Critic (SAC) and Twin Delayed Deep Deterministic Policy Gradient (TD3). These two algorithms take fundamentally different approaches to exploration — SAC uses a stochastic policy guided by entropy maximisation, while TD3 uses a deterministic policy with a scheduled Gaussian noise signal — making their comparison informative about the role of exploration in this type of task. Both are trained for 20,000 environment steps and evaluated over 100 test episodes. The RL results are also compared against a traditional sampling-based planner (RRT) to highlight the trade-offs between model-free and model-based approaches. SAC is found to converge faster and more stably, and the two algorithms show qualitatively different failure modes that reflect the limitations of their respective exploration strategies.

2 Method

2.1 Problem Formulation as a Markov Decision Process

The grasping task is formulated as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$. At each time step t, the agent observes state $s_t \in \mathcal{S}$, selects action $a_t \in \mathcal{A}$, transitions to s_{t+1} according to the physics simulator $\mathcal{T}$, and receives a scalar reward r_t.

State space. The observation is a four-dimensional vector:

$$s_t = [x_{\text{cube}},\ y_{\text{cube}},\ x_{\text{eef}},\ y_{\text{eef}}] \in \mathbb{R}^4, \tag{1}$$

where $(x_{\text{cube}}, y_{\text{cube}})$ is the horizontal position of the target cube and $(x_{\text{eef}}, y_{\text{eef}})$ is the end-effector (EEF) position projected onto the table plane. Including the EEF position gives the agent pro-

prioceptive feedback, which is necessary for learning accurate value estimates and for stable training.

Action space. The action is the desired horizontal position of the EEF:

$$a_t = [x_{\text{target}},\, y_{\text{target}}], \quad x_{\text{target}} \in [0.3,\, 0.7]\ \text{m}, \quad y_{\text{target}} \in [-0.3,\, 0.3]\ \text{m}. \tag{2}$$

The vertical height is fixed during the reaching phase at $z = 0.88$ m, reducing the problem to a 2-D reachability task. Inverse kinematics (IK) converts the desired Cartesian target to joint angles, which are tracked by PD controllers on each of the UR5's six joints.

Termination. An episode ends when the EEF comes within 1 cm of the cube centre, indicating task success, or when the step count exceeds $T_{\max} = 100$.

2.2 Simulation Environment

The environment is implemented in PyBullet using the Gymnasium interface. The physics simulation runs at $\Delta t = 1/300$ s. The scene consists of a flat table, a blue target cube measuring $5 \times 5 \times 5$ cm, and the UR5–Robotiq 85 robot (whose robotic arm modelling was modified based on an existing public implementation).

At the beginning of each episode the cube is placed at a randomly sampled position on a discrete grid:

$$x_{\text{cube}} \in \{0.4, 0.6\}\ \text{m}, \quad y_{\text{cube}} \in \{-0.3, -0.1, 0.1, 0.3\}\ \text{m}. \tag{3}$$

The robot arm is reset to a predefined neutral pose with joint angles $[0, -\pi/2, \pi/2, -1.5, -\pi/2, 0]$ and the gripper is fully opened to 0.085 m.

Grasping mechanics. Once the EEF is within the success radius, the arm descends to $z = 0.80$ m and the gripper closes incrementally. Closure terminates early when both finger-pad links simultaneously exert a contact force exceeding 3 N on the cube. The cube is then lifted to $z = 1.0$ m; a successful lift is confirmed if the cube's final z-coordinate exceeds 0.80 m.

Gripper friction. High lateral friction with $\mu = 1000$ and spinning friction with $\mu_s = 1.0$ are applied to the finger pads to ensure stable grasps once contact is established.

2.3 Reward Function

The reward function combines a **potential-based shaping** term with sparse goal bonuses:

$$r_t = \begin{cases} 80 + \max(0,\ T_{\max} - t) + r_{\text{grasp}} & \text{if EEF reached cube} \\ -10 \cdot d_t + \phi(s_t, s_{t-1}) & \text{otherwise} \end{cases} \tag{4}$$

where $d_t = \|[x_{\text{eef}}, y_{\text{eef}}] - [x_{\text{cube}}, y_{\text{cube}}]\|_2$ is the current reach distance, and the potential-based improvement bonus is:

$$\phi(s_t, s_{t-1}) = 8 \cdot \max\Big(0,\ d_{t-1} - d_t\Big). \tag{5}$$

The grasp component r_{grasp} is $+120$ for a successful lift, -20 if the gripper closes but fails to raise the cube, or -40 if no finger contact is made. The potential-based term provides a continuous learning signal that steers the agent toward the cube without changing the optimal policy, while the sparse bonuses reward the two distinct sub-tasks of reaching and grasping.

2.4 Algorithm Details

Soft Actor-Critic (SAC)

SAC is an off-policy algorithm that trains the agent to maximise both cumulative reward and the entropy of its policy:

$$\pi^* = \arg\max_{\pi} \sum_t \mathbb{E}_{(s_t,a_t)\sim\pi}\Big[r(s_t,a_t) + \alpha\,\mathcal{H}(\pi(\cdot|s_t))\Big], \tag{6}$$

where α is a temperature parameter balancing exploration against exploitation. In our implementation, α is automatically tuned to maintain a target entropy of $-\dim(\mathcal{A}) = -2$.
SAC trains a stochastic actor $\pi_\theta(a|s)$ alongside two Q-functions Q_{ψ_1}, Q_{ψ_2} that together reduce overestimation bias. The policy is updated to maximise Q-value while also being penalised for low entropy:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D}}\Big[\alpha \log \pi_\theta(a|s) - \min_{i=1,2} Q_{\psi_i}(s,a)\Big]. \tag{7}$$

This entropy bonus encourages the agent to explore broadly, which is especially helpful in tasks with sparse rewards or multiple discrete goal locations.

Twin Delayed DDPG (TD3)

TD3 is a deterministic off-policy algorithm that improves on DDPG with three additions: twin critic networks to reduce overestimation bias, delayed policy updates every $d = 2$ critic steps, and target policy smoothing. Exploration is achieved by adding Gaussian noise to actions during training:

$$a_t = \mu_\theta(s_t) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_t^2 I). \tag{8}$$

The noise schedule for σ_t is a critical design choice. If noise is fixed or poorly tuned, the deterministic policy can collapse to a single repetitive action. We use a linear decay schedule:

$$\sigma_t = \sigma_0 + \frac{t}{T}(\sigma_T - \sigma_0), \quad \sigma_0 = 0.08, \quad \sigma_T = 0.005, \tag{9}$$

reducing the standard deviation from about 1.6 cm at the start of training to 0.1 cm at the end. The target policy smoothing noise is set to 0.02 and clipped at ± 0.05, which is well below the 1 cm reach threshold and therefore does not affect the Q-value estimates.

2.5 Network Architecture and Training

Both algorithms share a two-layer MLP backbone implemented via Stable-Baselines3:

$$\mathbb{R}^4 \xrightarrow{\text{FC+ReLU}} 256 \xrightarrow{\text{FC+ReLU}} \mathbb{R}^2.$$

As illustrated in Figure 1, the observation $s_t = (x_{\text{cube}}, y_{\text{cube}}, x_{\text{eef}}, y_{\text{eef}})$ is fed into the shared backbone, after which the two algorithms diverge: the SAC actor appends a stochastic head that outputs a mean μ and log-standard-deviation σ, from which actions are sampled via the

reparameterisation trick using a squashed Gaussian; the TD3 actor instead produces a single deterministic action vector. The resulting action $a_t = [x_{\text{target}}, y_{\text{target}}]$ is sent to the environment, which executes the command, returns a reward r_t, and transitions to the next state s_{t+1}. Both algorithms are trained off-policy using a shared replay buffer that stores transitions (s_t, a_t, r_t, s_{t+1}); mini-batches are sampled uniformly to update the networks. All shared hyperparameters are kept identical to allow a fair comparison. Table 1 lists the full configuration.

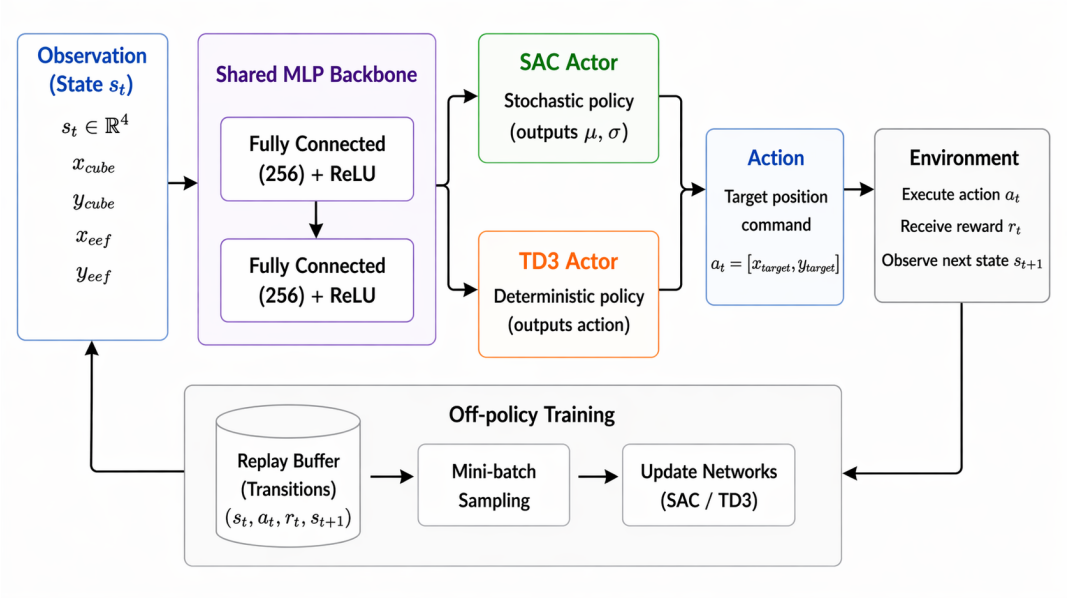

Figure 1: Overall training framework.

Table 1: Hyperparameters

Hyperparameter	**SAC**	**TD3**
Training steps	20,000	20,000
Replay buffer size	100,000	100,000
Batch size	256	256
Learning rate	3×10^{-4}	3×10^{-4}
Discount factor γ	0.99	0.99
Soft update τ	0.005	0.005
Gradient steps per env step	2	2
Learning starts	500	500
Hidden layer size	$[256, 256]$	$[256, 256]$
Policy delay	—	2
Target policy noise	—	0.02
Entropy coeff. α	auto	—
Exploration noise σ	stochastic	$0.08 \rightarrow 0.005$, linear decay

The last four rows capture fundamental algorithmic differences. Policy delay and target policy

noise are unique to TD3: delaying the actor update prevents the policy from overfitting to an immature critic, while target policy smoothing regularises the Q-function by adding small noise to target actions during Bellman backups. The entropy coefficient α is specific to SAC, where it controls the balance between reward and entropy and is automatically tuned to a target entropy of -2. The exploration mechanism differs structurally: SAC's stochastic actor generates diverse actions through sampling and needs no external noise source, whereas TD3's deterministic actor relies entirely on the linearly decaying additive noise to cover the workspace.

3 Results

3.1 Training Dynamics

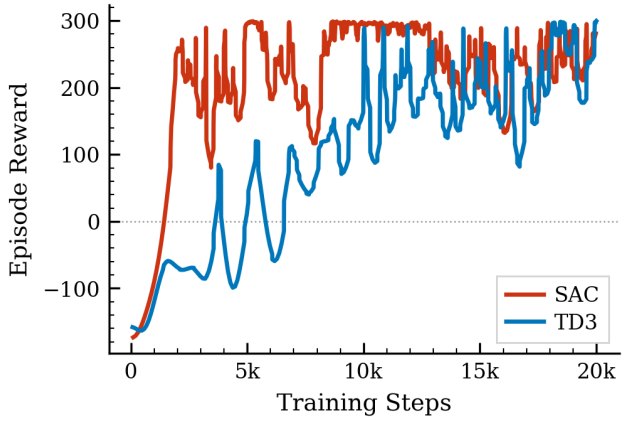

Figure 2: Episode reward during training.

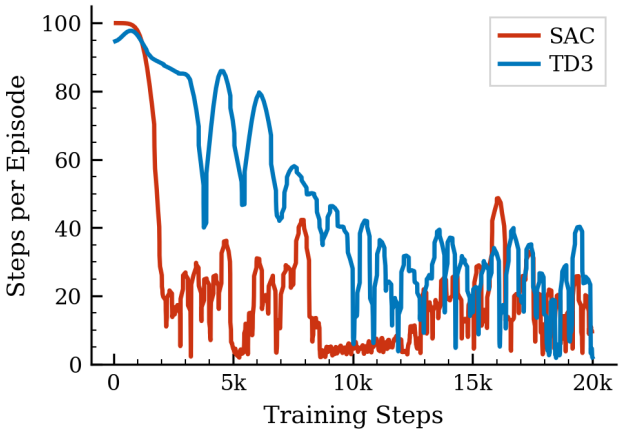

Figure 3: Episode length during training.

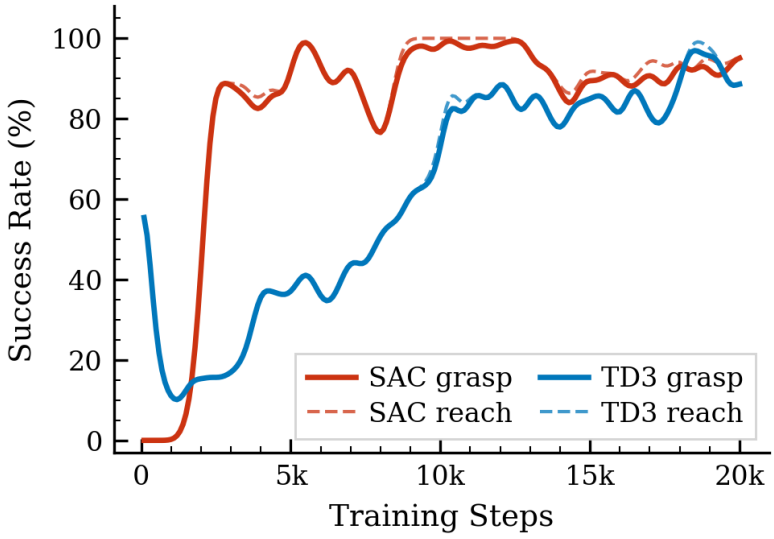

Figure 4: Rolling grasp success rate during training.

Figure 2 shows the smoothed episode reward and episode length curves; Figure 4 shows the rolling grasp success rate. Episode length is a useful indicator of progress: a successful episode ends as soon as the EEF reaches the cube, so shorter episodes mean faster and more frequent reaching, while episodes that hit the $T_{\max} = 100$ limit indicate consistent failure.

SAC. Training follows three clear phases. In the first 1,400 steps, the agent fails on every episode. The first 500 steps are used to fill the replay buffer with random actions; even after gradient updates begin, the policy is still near-random and rarely reaches the cube within the time limit. Around step 1,500 the first successful grasps appear, and by step 3,000 the rolling grasp rate reaches 80–90%. From that point it stays in the 80–97% range for the rest of training, ending at 96.7% at step 20,000.

SAC's fast convergence comes from three mutually reinforcing properties. First, the entropy term in the objective directly rewards action diversity, which pushes the agent to cover all eight discrete cube positions early in training rather than concentrating on the easiest ones. Second, the automatic tuning of α keeps the exploration level appropriate throughout: high when the policy is uncertain, and gradually lower as the policy improves, so no manual schedule is needed. Third, because the actor is stochastic, the same state produces different actions on different visits, filling the replay buffer with varied transitions without requiring any external noise.

TD3. TD3's training curve looks quite different. For the first 10,000 steps, the grasp rate stays low and oscillates between roughly 10% and 67% with no clear upward trend. Around step 10,000 there is an abrupt jump to 80–90% success, but the rate then continues to swing between 73% and 93%, reaching 100% briefly at step 18,200 before dropping again, and only stabilising near 100% in the final evaluation window.

The root cause is that TD3's deterministic policy has no built-in source of exploration — all diversity must come from the external noise $\epsilon \sim \mathcal{N}(0, \sigma_t^2 I)$. As σ_t decays from 0.08 to 0.005 over 20,000 steps, the policy becomes progressively more rigid. During the high-noise phase the arm covers the workspace broadly, but unevenly: positions close to the robot's neutral pose are visited far more often by chance than distant or lateral ones. The replay buffer therefore accumulates an unbalanced set of experiences, and the Q-function becomes well-calibrated only for the frequently visited positions. Once the noise has decayed, the policy works well for those positions but fails on the underrepresented ones where Q-estimates are inaccurate, producing the persistent oscillation.

The jump around step 10,000 marks the point at which the replay buffer has accumulated enough diverse successful transitions for the Q-function to generalise across most positions. However, the noise continues to decay at the same time, leaving only a narrow window to correct any remaining coverage gaps. Unlike SAC, which continuously rebalances exploration through its entropy objective, TD3 has no mechanism to return to underexplored positions once the noise is gone — those gaps become permanent.

3.2 Final Evaluation

After training, both models are evaluated in deterministic mode over 100 independent test episodes, with cube positions drawn uniformly from the full grid of eight configurations. This ensures that the evaluation reflects performance across the entire workspace, not just on easy

positions. Results are summarised in Table 2.

Table 2: Final evaluation results.

Metric	SAC	TD3
Reach Rate	**100%**	82%
Grasp Rate	**97%**	82%

SAC achieves a reach rate of 100% and a grasp rate of 97%, showing near-perfect performance across all eight cube positions. The perfect reach rate confirms that the stochastic policy covered the full workspace during training: every position accumulated enough successful transitions for reliable generalisation at test time. The 3-point gap between reach and grasp is a minor contact-geometry issue. In roughly 3 out of 100 episodes, the EEF arrives at the cube from a slightly off-centre angle near the boundary of the 1 cm success radius, where the finger pads do not make clean contact. This failure is unrelated to navigation quality and could be addressed by tightening the success radius or adding an approach-angle component to the reward.

TD3 achieves 82% for both reach and grasp. The fact that these two rates are identical is informative: every time TD3's deterministic policy successfully reaches the cube, the gripper closure succeeds without fail. The grasping motion itself has been well learned — the failures are purely navigational. The 18% miss rate corresponds directly to the coverage gaps left by the decaying noise schedule: those positions did not receive enough successful replay transitions for the Q-function to learn accurate values, and the deterministic test policy has no way to recover from these blind spots. The contrast with SAC's 100% reach rate clearly illustrates the cost of relying on a fixed external noise schedule rather than a self-adjusting exploration mechanism.

3.3 Comparison of SAC and TD3

Both SAC and TD3 are off-policy actor-critic algorithms, but their exploration strategies differ fundamentally. Table 3 summarises the key distinctions.

Table 3: SAC vs TD3

Property	SAC	TD3
Policy type	Stochastic, Gaussian	Deterministic
Exploration	Entropy maximisation	Additive Gaussian noise
Sample efficiency	Higher	Lower
Sensitivity to noise tuning	Low, auto-tuned α	High, σ schedule critical
Final reach / grasp rate	**100% / 97%**	82% / 82%
Convergence speed	~3,000 steps	~10,000 steps
Training stability	Stable after convergence	Persistent oscillation

The performance gap between SAC and TD3 comes down to a single structural difference: how

exploration is generated. SAC builds exploration directly into the policy objective through the entropy term, so the agent is naturally motivated to visit diverse states throughout training. This has three practical benefits in this task. First, all eight cube positions receive roughly equal coverage, resulting in a balanced replay buffer and well-calibrated Q-values across the workspace. Second, because α is automatically tuned, the level of exploration always matches the current stage of training — high when the policy is immature, and gradually lower as it improves. Third, the stochastic policy does not easily collapse: even after the grasp rate plateaus, the entropy term keeps the agent exploring rather than committing to a fixed set of trajectories.

TD3's exploration is entirely external and time-limited. A decaying noise schedule is necessary — large noise is needed early for broad coverage, but must reduce to near zero by the end of training so the deterministic policy can achieve the required 1 cm precision. This creates a fundamental tension: the schedule must be slow enough to cover the full workspace, but fast enough to allow fine-grained control before training ends. In practice this is difficult to balance perfectly. Some positions inevitably receive insufficient coverage before the noise drops too low to reach them reliably, and once the noise is gone, there is no way to correct the imbalance. The oscillation seen from step 10,000 onward is a direct consequence of this. More training steps would not help — the coverage gaps were formed during the early high-noise phase and cannot be undone. It is worth noting that once TD3 does reach the cube, its gripper closure is perfect, showing that the grasping sub-task itself was learned effectively.

4 Discussion

4.1 Soft Actor-Critic

SAC's maximum-entropy objective provides built-in, self-regulating exploration: the entropy term encourages the policy to remain stochastic and cover the full workspace without any manually designed noise schedule. The automatic tuning of α keeps the exploration level appropriate throughout training — high when the policy is uncertain, and gradually lower as it matures — removing a hyperparameter that would otherwise require careful per-environment adjustment. These properties together allow SAC to converge quickly and maintain stable performance once a capable policy is found.

The main limitation of SAC stems from the same stochasticity that drives its exploration. Even a well-trained Gaussian policy occasionally selects a slightly off-centre approach angle, causing the gripper to close without making clean contact. This produces a small but consistent failure rate at test time. The issue could be mitigated by tightening the success radius, adding an approach-angle reward term, or switching to a deterministic policy at inference while keeping the stochastic actor for training.

4.2 Twin Delayed DDPG

TD3's deterministic policy is noise-free at test time, which in principle allows precise and repeatable motor commands. The twin-critic architecture and delayed policy updates reduce

overestimation bias and stabilise learning, making TD3 a theoretically well-motivated choice for continuous control tasks.
In practice, however, all exploration must come from an external noise schedule, which creates a fundamental tension: noise must be large enough early on to cover the workspace, yet small enough by the end to achieve the required 1 cm precision. Getting this balance right is difficult without prior task knowledge. If the schedule is imperfect, some cube positions receive insufficient coverage before the noise drops too low, leaving permanent blind spots that the deterministic test policy cannot recover from. This explains the persistent oscillations and lower final performance compared to SAC.

5 Conclusion

Challenges encountered. The most significant challenge was the long early failure phase — about 1,400 steps for SAC and 10,000 steps for TD3 — during which the agent receives mostly negative rewards and makes little visible progress. Potential-based reward shaping was essential in shortening and stabilising this phase by providing a continuous guidance signal rather than waiting for a rare successful episode. For TD3, tuning the noise decay schedule was also non-trivial: too fast, and the policy converges before covering the workspace; too slow, and it never achieves the precision needed for a clean grasp.
Lessons learned. First, reward shaping can matter as much as algorithm choice in sparse-reward manipulation tasks: without the potential-based term, training was considerably longer and less stable. Second, SAC's entropy regularisation and automatic temperature tuning made it noticeably more robust and sample-efficient than TD3 under the same training budget. Third, even a simplified 2-D action space can yield useful grasp policies, as long as the reward separately accounts for the reaching and grasping stages.
Future directions. Expanding the action space to 6-D pose control would let the agent handle objects at different heights and orientations. Replacing the discrete cube grid with continuous random placement would test true generalisation. Hindsight Experience Replay (HER) could improve sample efficiency in the sparse-reward setting. Domain randomisation would help reduce the sim-to-real gap. Finally, adding depth images to the observation space would allow the agent to handle objects of varying shape and size.

References

[1] J. Bohg, A. Morales, T. Asfour, and D. Kragic, "Data-driven grasp synthesis — a survey," *IEEE Transactions on Robotics*, vol. 30, no. 2, pp. 289–309, 2014. pages 1

[2] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. A. Ojea, and K. Goldberg, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," *arXiv preprint arXiv:1703.09312*, 2017. pages 1

[3] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, "Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection," *The International Journal of Robotics Research*, vol. 37, no. 4–5, pp. 421–436, 2018. pages 1

[4] L. Pinto and A. Gupta, "Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3406–3413, 2016. pages 1

[5] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, "Scalable deep reinforcement learning for vision-based robotic manipulation," in *Proc. Conference on Robot Learning (CoRL)*, pp. 651–673, 2018. pages 1